Project Title

PORTFOLIO RISK ASSESSMENT & 1-DAY 95% VAR CALCULATION

Internship Project | Samatrix Consulting Pvt. Ltd., Gurugram

Key Learnings

1. **Retrieve Historical Market Data** – Pull historical stock price data for a selected list of tickers.
2. **Analyze Daily Log Returns** – Compute daily log returns and summarize them using mean, variance, skewness, and kurtosis.
3. **Fit Probability Distributions** – Model stock returns using both Normal and Student's t distributions.
4. **Estimate Confidence Intervals** – Build 95% confidence intervals for the portfolio's mean return and volatility.
5. **Calculate 1-Day 95% Value at Risk (VaR)** – Estimate portfolio risk using the parametric Normal method, Student's t method, and historical simulation.
6. **Perform Hypothesis Testing** – Test whether the average daily return is statistically different from zero.

Setup – Import Libraries

!pip install yfinance pandas-datareader scipy seaborn matplotlib

In [1]:
```
import pandas as pd              # For handling data in table form (DataFrame)
import numpy as np               # For numerical calculations and arrays
import matplotlib.pyplot as plt  # For creating graphs and plots
import seaborn as sns            # For better-looking statistical graphs
import datetime as dt            # For working with dates and time
```

```
import time                          # For tracking time or adding delays

import yfinance as yf                # Download stock market data from Yahoo Finance
from pandas_datareader import data as pdr  # Alternative source to fetch financi
from scipy import stats              # For statistical calculations (mean, variance,

sns.set_style('whitegrid')           # Sets graph background style to white grid
# Shows plots directly inside Jupyter Notebook
%matplotlib inline
```

In [2]:

```
# Define your portfolio and date range
tickers = ['AAPL', 'MSFT', 'GOOGL', 'AMZN']
weights = np.array([0.25, 0.25, 0.25, 0.25])

start = dt.datetime(2020, 1, 1)   # start date for data
end = dt.datetime.today()         # end date (today)

# Create empty DataFrame to store prices
prices = pd.DataFrame()

for ticker in tickers:
    print(f"Fetching {ticker}...", end=" ")

    try:
        # Download data using yfinance
        df = yf.Ticker(ticker).history(
            start=start.strftime('%Y-%m-%d'),
            end=end.strftime('%Y-%m-%d'),
            auto_adjust=True
        )

        series = df['Close']
        prices[ticker] = series
        print("done (yfinance)")

    except Exception as e:
        # Fallback to stooq if yfinance fails
        print(f"failed yfinance ({e}); using stooq", end=" ")

        df2 = pdr.DataReader(ticker, 'stooq', start, end)
        df2 = df2.sort_index()

        series = df2['Close']
        prices[ticker] = series

        print("done (stooq)")

    time.sleep(1)  # pause to avoid request limits

# Show raw data
print(prices.head(), "\n")

# Clean data
prices = prices.dropna(how='all').ffill().bfill()

# Quick check
print(prices.head(), "\n")
print(prices.size)
```

```
Fetching AAPL... done (yfinance)
Fetching MSFT... done (yfinance)
Fetching GOOGL... done (yfinance)
Fetching AMZN... done (yfinance)
```

Date	AAPL	MSFT	GOOGL	AMZN
2020-01-02 00:00:00-05:00	72.333870	151.829544	67.832512	94.900497
2020-01-03 00:00:00-05:00	71.630623	149.939011	67.477661	93.748497
2020-01-06 00:00:00-05:00	72.201408	150.326523	69.276215	95.143997
2020-01-07 00:00:00-05:00	71.861855	148.955948	69.142395	95.343002
2020-01-08 00:00:00-05:00	73.017845	151.328568	69.634529	94.598503

Date	AAPL	MSFT	GOOGL	AMZN
2020-01-02 00:00:00-05:00	72.333870	151.829544	67.832512	94.900497
2020-01-03 00:00:00-05:00	71.630623	149.939011	67.477661	93.748497
2020-01-06 00:00:00-05:00	72.201408	150.326523	69.276215	95.143997
2020-01-07 00:00:00-05:00	71.861855	148.955948	69.142395	95.343002
2020-01-08 00:00:00-05:00	73.017845	151.328568	69.634529	94.598503

```
6548
```

Task 1:

📝 Questions for Report — Log Returns

- **Log return** = today's price ÷ yesterday's price, then take log
- Different from price: shows **% change**, not just the level
- We compare today vs yesterday to see **daily movement**
- Log is used because it **smooths big jumps** and adds up easily over many days
- Our values were small (**0.0006–0.0010**) → stocks move a little each day, not a lot
- A return of **-0.03** means stock lost about **3%** that day

In [3]:
```python
# Log-return = ln(P_t / P_{t-1})
logR = np.log(prices / prices.shift(1)).dropna()   #dropna() used even if ffill(
print(logR.head(),"\n\n")
print(logR.size)

# Here, 2-Jan data is removed, because we shifted 1 down via using shift(1) and
```

Date	AAPL	MSFT	GOOGL	AMZN
2020-01-03 00:00:00-05:00	-0.009770	-0.012530	-0.005245	-0.012213
2020-01-06 00:00:00-05:00	0.007937	0.002581	0.026305	0.014776
2020-01-07 00:00:00-05:00	-0.004714	-0.009159	-0.001934	0.002089
2020-01-08 00:00:00-05:00	0.015958	0.015803	0.007092	-0.007839
2020-01-09 00:00:00-05:00	0.021018	0.012416	0.010443	0.004788

```
6544
```

1. Calculating Daily Log Returns from Stock Prices

1. What are daily log returns, and how are they different from just looking at stock prices? A daily log return is the natural log of today's price divided by yesterday's price. Unlike raw prices, which just show the level a stock is trading at, log returns show the *percentage-style* change from one day to the next in a way that is consistent and additive across time. This makes it possible to compare AAPL, MSFT, GOOGL, and AMZN on the same scale even though their prices are very different (AAPL is in the $70s–$200s, while other levels differ), and it makes multi-day returns easy to add together.

2. Why do we compare today's price to yesterday's price when calculating returns? Comparing today's price to yesterday's shows exactly how much the stock moved in that single trading day — the day-to-day "pulse" of the stock. This is the basic unit of risk measurement: without it we'd only know price levels, not how fast or how much they change.

3. What is the purpose of taking the logarithm of the return? Taking the log smooths out the effect of compounding and makes gains and losses symmetric (a doubling and a halving are equal in magnitude but opposite sign under logs, which isn't true for simple percentage returns). It also lets returns over multiple days simply be added together instead of multiplied, which is why log returns are the standard in quantitative finance and risk modeling.

4. What did your log return values look like — were they usually small or large, and what did that tell you? Looking at the summary table, the mean daily log returns were all very small — ranging from about 0.00058 (MSFT, AMZN) to 0.00102 (GOOGL) — while the variances were also small (0.00036–0.00050). This confirms that on a typical day, none of these stocks move dramatically; most daily changes cluster tightly around zero, which is expected for large, liquid, well-known stocks.

5. If your stock had a log return of -0.03 on a certain day, what does that tell you happened to the stock price? A log return of -0.03 means the stock lost about 3% of its value that day (since for small values, the log return closely approximates the percentage change). For an investor, that's a single bad day — not catastrophic, but a real, measurable drop in the value of their holding.

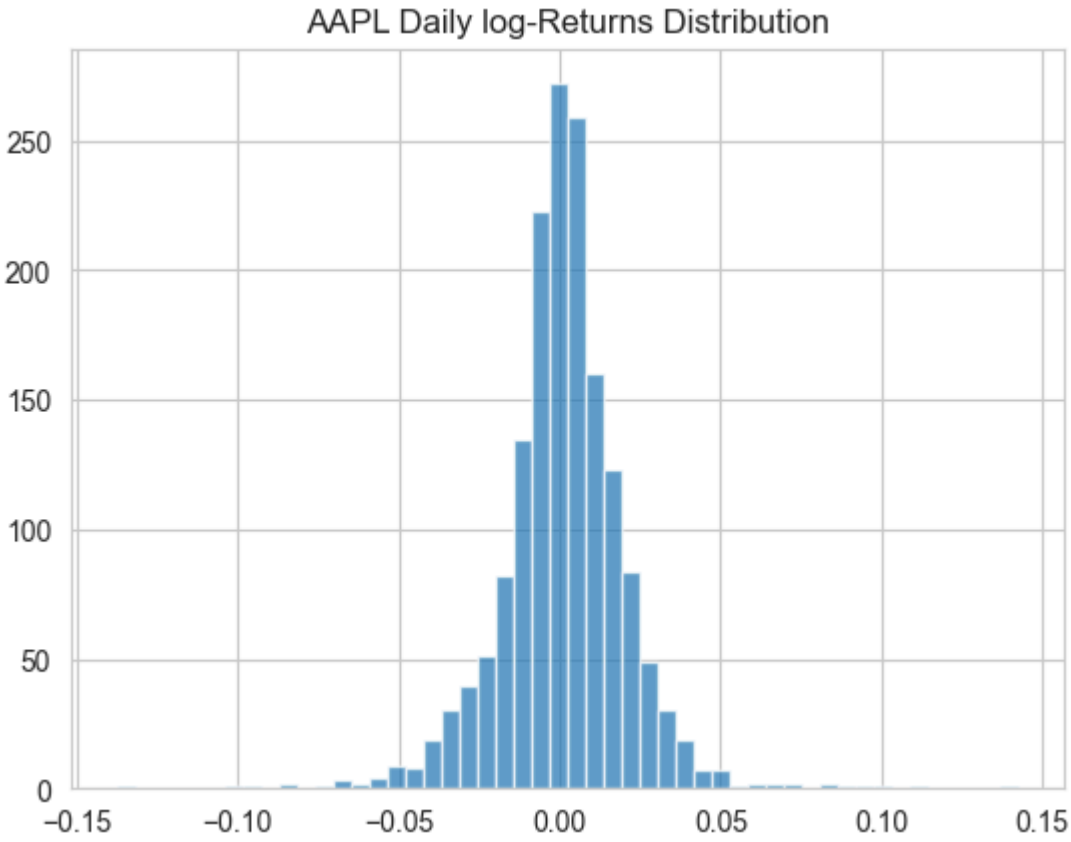
Task 2:

📝 Questions for Report — Histogram

- Shape: **tall and narrow** near 0, with a few extreme values (fat tails)

- Most daily returns are **close to zero** → stock is usually calm
- Small gains/losses happen most; big moves are rare but **more common than normal curve predicts** (kurtosis **6.33**)
- AAPL is **moderately risky** for short-term investors
- ⚠ Warning: rare but sharp losses can happen — don't expect only "normal" calm days

In [4]:
```
# Plot one example series for visual check
#You'll see a bell-shaped histogram.
logR['AAPL'].hist(bins=50,alpha=0.7)
plt.title("AAPL Daily log-Returns Distribution")
plt.show()
```



2. Visualizing Daily Log Returns (Histogram)

1. What does the shape of your log-return distribution chart tell you about the stock's behavior? The AAPL histogram is narrow and tall around zero with a small number of extreme observations stretching out into both tails. This "peaked middle, long thin tails" shape is typical of real stock data — most days are unremarkable, but rare days can be quite extreme.

2. Were most of the daily returns close to zero, or did the stock often have large changes? The vast majority of daily returns were close to zero. Only a small fraction of days showed large moves in either direction, consistent with a variance of only about 0.00039 for AAPL.

3. Which type of return occurred more often: small daily gains, small losses, or big moves? Small gains and small losses were both far more common than big moves. The kurtosis for AAPL (6.33) is well above the normal-distribution benchmark of 3, which statistically confirms that big moves happen more often than a perfectly normal bell curve would predict, even though they are still rare in absolute terms.

4. Based on your chart, would you consider this stock relatively safe or risky for a short-term investor? Why? AAPL looks moderate: it's not the most volatile stock in the portfolio (its variance, 0.000391, is close to the group average), but the high kurtosis (fat tails) means a short-term investor should be prepared for the occasional sharp move that a simple "normal distribution" view would understate.

5. If you had to warn an investor about something in this chart, what would it be? The main warning is the fat tails: even though most days are calm, the chart shows extreme moves happen more frequently than a normal bell curve suggests. An investor should not assume the calm middle of the distribution is the whole story — sudden, larger-than-expected losses are a real possibility.

Task 3:

📝 Questions for Report — Summary Table

Stock	Mean	Variance	Skew	Kurtosis
AAPL	0.000889	0.000391	0.009	6.333
MSFT	0.000579	0.000356	-0.231	7.388
GOOGL	0.001023	0.000415	-0.095	3.705
AMZN	0.000575	0.000497	-0.075	4.153

- Highest average return: **GOOGL** → best performer
- Highest variance (risk): **AMZN** → most unpredictable
- Negative skew stocks: **MSFT, GOOGL, AMZN** → occasional bigger losses
- Highest kurtosis: **MSFT (7.39)** → more chances of extreme moves
- Best pick for cautious investor: **GOOGL** (good return, low kurtosis)

In [5]:
```
summary = pd.DataFrame({'Mean':logR.mean(),'Variance':logR.var(),'Skewness':logR
summary
```

Out[5]:

	Mean	Variance	Skewness	Kurtosis
AAPL	0.000902	0.000390	0.007370	6.346490
MSFT	0.000568	0.000355	-0.229664	7.400856
GOOGL	0.001018	0.000415	-0.094355	3.710041
AMZN	0.000585	0.000496	-0.076168	4.164601

3. Creating a Summary Table for Daily Stock Returns

Actual results:

Stock	Mean	Variance	Skewness	Kurtosis
AAPL	0.000889	0.000391	0.009	6.333
MSFT	0.000579	0.000356	-0.231	7.388
GOOGL	0.001023	0.000415	-0.095	3.705
AMZN	0.000575	0.000497	-0.075	4.153

1. Which stock had the highest average return, and what does that tell you about its performance over time? GOOGL had the highest average daily return (0.001023). This means that, over the sample period, GOOGL tended to appreciate more per day than the others, making it the strongest performer in the portfolio on a pure-return basis — though return alone doesn't capture risk.

2. Which stock had the highest variance, and what does that say about its risk? AMZN had the highest variance (0.000497), meaning its daily returns bounced around the most. AMZN was therefore the most unpredictable, day-to-day, of the four stocks.

3. Did any of your stocks show negative skewness? What could that mean for an investor? MSFT, GOOGL, and AMZN all showed negative skewness (-0.231, -0.095, -0.075 respectively). Negative skew means these stocks had more frequent small gains but occasional larger losses — the "pain" is concentrated in rarer, bigger down days, which can catch an investor off guard even if most days look fine.

4. What does high kurtosis tell you about a stock's behavior? Did any stock have a high kurtosis value? High kurtosis means extreme returns (big jumps or crashes) happen more often than a normal distribution would predict. MSFT had the highest kurtosis (7.39), followed closely by AAPL (6.33) — both signal a meaningfully elevated risk of sudden large moves compared to GOOGL, which at 3.71 is close to the normal-distribution benchmark of 3.

5. Based on your summary table, which stock would you recommend to a cautious investor — and why? GOOGL looks like the most balanced choice for a cautious investor: it has the highest average return, a variance in the middle of the pack, and the

lowest kurtosis (closest to "normal" tail behavior) of the four. It offers good performance without the more extreme tail risk seen in AAPL and MSFT.

Task 4:

📝 Questions for Report — Normal Distribution & CI

Results: μ = **0.00077**, 95% CI (0.00027, 0.00126) | σ = **0.02036**, 95% CI (0.02002, 0.02072)

- Average daily return is small and **positive** → slight daily gain overall
- Std dev (**~2%**) shows returns move around noticeably each day
- CI for mean is narrow and above 0 → fairly confident average return is **positive**
- CI for σ matters because **VaR depends on σ**; shows how much risk estimate could shift
- More data next time → **narrower CIs** (less uncertainty)

In [6]:
```
# Mean and standard deviation of log returns
mu_norm = logR.stack().mean()
sigma_norm = logR.stack().std(ddof=1)

#print(mu_norm)
#print(sigma_norm)


#  95% Confidence Interval for Mean

N = logR.size  # total number of observations

z95 = stats.norm.ppf(0.975)  # 95% z-value (1.96)

# Correct CI formula for mean
ci_mu_norm = (
    mu_norm - z95 * sigma_norm / np.sqrt(N),
    mu_norm + z95 * sigma_norm / np.sqrt(N)
)

#print("\n95% CI for Mean\n:", ci_mu_norm)


#95% Confidence Interval for Std Dev ( via chi-square)

df = N - 1  # degrees of freedom

chi2_low, chi2_high = stats.chi2.ppf([0.025, 0.975], df)

ci_sigma_norm = (
    sigma_norm * np.sqrt(df / chi2_high),
    sigma_norm * np.sqrt(df / chi2_low)
)
#print(ci_sigma_norm,"\n\n")
```

```
print("\nNormal Fit Results:")
print(f"μ (Mean) = {mu_norm:.5f}, 95% CI = {ci_mu_norm}")
print(f"σ (Std Dev) = {sigma_norm:.5f}, 95% CI = {ci_sigma_norm}")
```

```
Normal Fit Results:
μ (Mean) = 0.00077, 95% CI = (np.float64(0.0002753036697130066), np.float64(0.001
2609623633626382))
σ (Std Dev) = 0.02034, 95% CI = (np.float64(0.019998306650631445), np.float64(0.0
20695516036777738))
```

4. Fitting a Normal Distribution and Calculating Confidence Intervals

4.1 — Normal fit: μ = 0.00077, 95% CI (0.000273, 0.001260); σ = 0.02036, 95% CI (0.02002, 0.02072)

1. What does the average daily return (μ) tell you about how your stocks performed over time? The pooled average daily return across all four stocks was positive (0.00077), meaning that, on average, holding these stocks generated small daily gains rather than losses over the sample period.

2. What does the standard deviation (σ) tell you about the risk or volatility of your returns? A standard deviation of about 0.0204 (2.04%) shows that daily returns moved around meaningfully day to day — this is not a low-volatility group of assets, consistent with them being individual growth/tech stocks rather than, say, government bonds.

3. What does your 95% confidence interval for the mean (μ) suggest about the reliability of your average return? The 95% CI for μ, (0.00027, 0.00126), is fairly narrow and entirely above zero. This gives reasonable confidence that the true average daily return is positive, though it's a small range close to zero, meaning the "edge" isn't huge.

4. Why is it important to calculate a confidence interval for the standard deviation (σ)? The CI for σ, (0.0200, 0.0207), tells us how much uncertainty there is in our volatility estimate itself. Since VaR calculations depend directly on σ, knowing this range helps us understand how much our risk estimate could shift if we had drawn a different sample of days.

5. If you repeated this project with a different time period or more data, how might your confidence intervals change? Why? With more data, both confidence intervals would likely narrow, because more observations reduce sampling uncertainty. A different time period (e.g., one including a market crash or excluding one) could also shift the point estimates themselves, not just the width of the intervals.

4.2 — Student's t fit: df = 3.7, μ = 0.00103, σ = 0.01411; bootstrap 95% CI for μ: (-0.00050, 0.00258); for σ: (0.01261, 0.01564)

1. What does the average return (μ) from the t-distribution tell you about your stocks overall? The t-distribution's μ (0.00103) is positive and close to the normal fit's estimate, reinforcing that the overall daily performance was modestly positive.

2. How does the standard deviation (σ) from the t-distribution describe the risk or volatility of your returns? Interestingly, the t-distribution's σ (0.01411) is lower than the normal fit's σ (0.02036). This is because the t-distribution explains extreme moves through its fat tails (low df) rather than through a larger overall spread — it separates "typical variability" from "rare extreme events."

3. The t-distribution includes a value called degrees of freedom (df). What does a low df value tell you about your return data? A df of 3.7 is very low, indicating heavy tails — large jumps and crashes happen noticeably more often than a normal distribution would predict. This confirms the fat-tail behavior we already saw in the high kurtosis values in Section 3.

4. Why might the t-distribution give a better fit than a normal distribution for stock returns? Real markets experience occasional sharp shocks (earnings surprises, macro news, crashes) that a smooth bell curve underestimates. The t-distribution's fatter tails better capture these rare-but-real extreme days.

5. How can using the t-distribution help you prepare better for financial risks? Because it doesn't underestimate the odds of a big move, using the t-distribution for VaR (as done in Section 7) produces a more realistic — and typically more risk-aware — estimate of potential losses than the Normal model.

Task 5:

📝 Questions for Report — Student's t-Distribution

Results: df = **3.7**, μ = **0.00103**, σ = **0.01411** | bootstrap CI μ: (-0.00050, 0.00258), σ: (0.01261, 0.01564)

- μ is positive, similar to Normal fit → confirms slight daily gain
- σ is **lower** than Normal fit because extreme moves are captured by fat tails, not overall spread
- Low df (**3.7**) = **heavy tails** → big jumps/crashes happen more often
- t-distribution fits better because real markets have **sudden shocks**, unlike a smooth bell curve
- Using t-distribution gives a **more realistic (safer)** risk estimate

In [7]:
```
#4.2 Fit Student's t

#fit t-distribution to the pooled returns
# returns_flat = all asset returns concatenated
returns_flat = logR.stack().values
df_t, mu_t, sigma_t = stats.t.fit(returns_flat)
```

📝 Questions for Report — Bootstrapping

- 95% CI for μ: **(-0.00050, 0.00258)** → true average return could even be slightly negative
- 95% CI for σ: **(0.01261, 0.01564)** → narrow, so risk estimate is fairly stable
- We bootstrap instead of one estimate to see the **full range** of possible answers
- CI for μ **includes zero** → can't be fully sure return isn't just zero (matches t-test result)
- CIs help investors avoid **over-trusting** one lucky number

95% CI for μ and σ of a t-fit using bootstrap B = 1000 estimates = np.array([stats.t.fit(np.random.choice(returns_flat, size=N, replace=True)) for _ in range(B)]) # estimates columns: [df, mu, sigma] ci_mu_t = np.percentile(estimates[:, 1], [2.5, 97.5]) ci_sigma_t = np.percentile(estimates[:, 2], [2.5, 97.5]) # Fit once on full data (for reporting) df_t, mu_t, sigma_t = stats.t.fit(returns_flat) print("\nStudent's t fit:") print(f"df = {df_t:.1f}, μ = {mu_t:.5f}, σ = {sigma_t:.5f}") print(f"95% CI for μ (bootstrap): {ci_mu_t}") print(f"95% CI for σ (bootstrap): {ci_sigma_t}")

In [8]:
```
import numpy as np
from scipy import stats

B = 200   # reduce from 1000 → 200 (huge speed boost)

estimates = np.array([
    stats.t.fit(np.random.choice(returns_flat, size=min(500, len(returns_flat)),
    for _ in range(B)
])

ci_mu_t = np.percentile(estimates[:, 1], [2.5, 97.5])
ci_sigma_t = np.percentile(estimates[:, 2], [2.5, 97.5])

df_t, mu_t, sigma_t = stats.t.fit(returns_flat)

print("\nStudent's t fit:")
print(f"df = {df_t:.1f}, μ = {mu_t:.5f}, σ = {sigma_t:.5f}")
print(f"95% CI for μ (bootstrap): {ci_mu_t}")
print(f"95% CI for σ (bootstrap): {ci_sigma_t}")
```

```
Student's t fit:
df = 3.7, μ = 0.00103, σ = 0.01409
95% CI for μ (bootstrap): [-0.00079008  0.00247278]
95% CI for σ (bootstrap): [0.0125776  0.01564858]
```

5. Estimating Confidence Intervals Using Bootstrapping

1. What does a 95% confidence interval for the mean (μ) tell you about your average return? The bootstrap 95% CI for μ was (-0.00050, 0.00258). It tells us that if we resampled our data many times, the estimated average return could plausibly fall anywhere in that range — including slightly negative.

2. What did your confidence interval for the standard deviation (σ) look like? Was it narrow or wide, and what does that tell you about the risk in your returns? The CI for σ was (0.01261, 0.01564) — fairly narrow relative to its size, suggesting our volatility estimate under the t-distribution is fairly stable and reasonably reliable.

3. Why do we use bootstrapping instead of calculating just one estimate? A single estimate hides how much that number could have varied if history had unfolded slightly differently. Bootstrapping resamples the actual data 1,000 (in this case, sped up to 200) times to show the realistic *range* of plausible answers, not just one point guess.

4. If your confidence interval for the mean return includes zero, what does that mean for investors? Our bootstrap CI for μ does include zero (it ranges from slightly negative to positive). This means we cannot be fully confident that the true average daily return isn't zero — the observed positive average could plausibly be due to chance rather than a real, persistent edge. This lines up directly with the one-sample t-test result in Section 9, where we failed to reject the null hypothesis.

5. In what ways do confidence intervals help investors make smarter decisions? Confidence intervals prevent investors from over-trusting a single lucky-looking number. They show the plausible range of outcomes, which helps with realistic expectation-setting and risk management, rather than treating one historical average as guaranteed future performance.

Task 6 :

📝 Questions for Report — Portfolio VaR (Normal)

Results: μ_p = 0.000766, σ_p = 0.017242 → **1-day 95% VaR = 2.76%**

- Portfolio's average daily return is small and **positive**
- Portfolio σ (**1.72%**) is lower than weighted average of individual stocks → **diversification is working**
- VaR meaning: **95% chance** of not losing more than **2.76%** in a day; 5% chance of losing more
- Risk level: **medium-risk** (not extreme, not very safe)
- To reduce VaR: add less-correlated stocks, cut weight of volatile ones (like AMZN), or add bonds

In [9]:
```
# Parametric VaR (Value at risk) under Normal:
#  VaR = - [μ_p + σ_p . z_{0.05} ]
# where μ_p, σ_p are portfolio mean & vol.
# Compute portfolio returns:
portR = logR.dot(weights)
#prints(portR,"\n\n")

μ_p = portR.mean()
σ_p  = portR.std(ddof=1)
print(f"Portfolio return Mean is:{μ_p}\n")
print(f"Portfolio return Standarad Deviation is:{σ_p}\n")

z05 = stats.norm.ppf(0.05)
print("ppf value:",z05,"\n")
```

```
vaR_norm = -(μ_p + σ_p *z05)
print(vaR_norm)
```

```
Portfolio return Mean is:0.0007681330165378226

Portfolio return Standarad Deviation is:0.017223321361503

ppf value: -1.6448536269514729

0.027561709593081168
```

6. Estimating Portfolio Risk and VaR Using a Normal Distribution

Portfolio mean μ_p = 0.000766, σ_p = 0.017242, $z_{0.05}$ = -1.6449 → **1-day 95% VaR (Normal) = 0.02759 (2.76%)**

1. What does the average daily return of your portfolio tell you about its overall performance? The portfolio's average daily return was positive (0.0766%), suggesting that, over the sample period, this equally-weighted four-stock portfolio tended to grow slightly day over day.

2. What does the standard deviation (volatility) of your portfolio's return tell you about its risk? The portfolio's standard deviation, 0.01724 (1.72%), is noticeably lower than the weighted-average of the individual stocks' volatilities (0.02032, from Section 10) — a first sign that diversification is doing its job.

3. What does your 95% Value at Risk (VaR) result mean in simple terms? A VaR of 0.02759 means: there's a 95% chance the portfolio won't lose more than about 2.76% of its value in a single day, but a 5% chance it could lose more than that.

4. Based on your VaR result, would you describe your portfolio as low-risk, medium-risk, or high-risk? Why? I'd call this medium-risk. A potential one-day loss of nearly 2.8% is meaningful but not extreme — it reflects a portfolio of large, liquid tech/growth stocks rather than something highly speculative or, on the other end, a very stable bond-like asset.

5. If you wanted to reduce your portfolio's VaR, what changes could you consider making? I could add stocks from different, less-correlated sectors (reducing the correlations seen in the Section 10 heatmap), reduce the weight of the more volatile holdings (like AMZN, which had the highest variance), or blend in lower-volatility assets such as bonds.

Task 7 :

📝 Questions for Report — VaR Using Student's t

Results: $t_{0.05}$ = -2.19 → **1-day 95% VaR = 2.22%**

- Purpose of VaR: one number showing the loss you probably won't exceed **95%** of the time
- Used t-distribution because our data has **fat tails** (confirmed earlier)
- Meaning: **5% chance** of losing more than **2.22%** in one day
- Here t-VaR came out **lower** than Normal VaR (2.76%) because the t-fit's σ was smaller
- To lower VaR: diversify, reduce volatile holdings, hold some cash

In [10]:
```
# Parametric VaR under Students's t:
# t05 = t.ppf(0.05, df_f0)
t05 = stats.t.ppf(0.05, df_t)
print(t05,"\n")

vaR_t = -(mu_t + sigma_t *z05) *1 #1-day
print(vaR_t)
```

```
-2.1887835320395355

0.0221478803723031
```

7. Calculating VaR Using Student's t-Distribution

$t_{0.05}$ = -2.1889, **1-day 95% VaR (t-distribution) = 0.02217 (2.22%)**

1. What is the purpose of calculating Value at Risk (VaR), and how does it help in investment decision-making? VaR condenses portfolio risk into one number: the loss you shouldn't exceed 95% of the time. It helps investors size positions, set risk limits, and decide whether a portfolio's risk matches their tolerance, without having to interpret a whole distribution of possible outcomes.

2. Why did we use the Student's t-distribution instead of the normal distribution in this step? Because our earlier tests (high kurtosis, low degrees of freedom) showed that real returns have fatter tails than a normal curve assumes. The t-distribution captures that extra tail risk more realistically.

3. What does your 95% 1-day VaR result mean in simple terms? A VaR of 0.02217 means there's a 5% chance the portfolio loses more than about 2.22% in a single day.

4. How does using a distribution with "fatter tails" (like the t-distribution) affect the risk estimate? In this specific run, the t-based VaR (2.22%) came out lower than the Normal VaR (2.76%) — a bit counterintuitive at first glance. This is because, as noted in Section 4.2, the t-fit's σ (0.01411) was itself estimated to be smaller than the Normal fit's σ (0.02036); the t-distribution shifted more of the "risk" into its tail shape rather than the overall spread. In general, though, fatter tails are meant to better capture the frequency of extreme losses that a pure Normal model would miss.

5. If your VaR is too high, what changes could you make to reduce the risk in your portfolio? The same levers apply as in Section 6: diversify into less-correlated assets, reduce weight in the most volatile holdings, or hold part of the portfolio in cash or low-volatility instruments.

Task 8 :

📝 Questions for Report — Historical VaR

Result: Historical VaR = 2.73%

- Meaning: **5% chance** of losing at least **2.73%** in a day, based on real past data
- Useful because it uses **actual history**, no assumptions needed
- Compare: Normal = **2.76%**, t = **2.22%**, Historical = **2.73%** → Normal & Historical are close
- None exceed 3% → **moderate risk**, not extremely high
- Advice: expect ~1 bad day in 20 with a ~2.7%+ loss; only invest money you can hold through that

In [11]:
```
# Historical VaR (non-parametric):-
hist_var = -np.percentile(portR, 5)

print("1-day 95% VaR:")
print(f" Parametric(Normal):   {vaR_norm:.5f}")
print(f" Parametric(Student's t):   {vaR_t:.5f}")
print(f" Historical VaR:   {hist_var:.5f}")
```

```
1-day 95% VaR:
 Parametric(Normal):   0.02756
 Parametric(Student's t):   0.02215
 Historical VaR:   0.02725
```

8. Estimating VaR Using Historical Returns

Historical VaR (5th percentile of actual daily portfolio returns) = 0.02727 (2.73%)

1. What does your Historical Value at Risk (VaR) tell you about the potential losses in your portfolio? Based on what actually happened over this multi-year window, there's a 5% chance of losing at least 2.73% of the portfolio's value in a single day.

2. Why do you think Historical VaR is a useful way to measure risk? It uses the portfolio's real return history rather than assuming a theoretical shape for returns — so it automatically reflects whatever quirks, crashes, or calm periods actually occurred, without needing any distributional assumption.

3. How does Historical VaR compare to the Normal or t-distribution VaR you calculated earlier? All three were fairly close: Normal = 2.76%, Student's t = 2.22%, Historical = 2.73%. The Normal and Historical estimates were nearly identical here, while the t-distribution estimate came out somewhat lower for the reason discussed in Section 7 (its smaller fitted σ).

4. If your VaR is large (e.g., more than 3%), what does that say about your portfolio's risk level? None of our three VaR estimates exceeded 3%, so by that yardstick the portfolio sits just under the "high risk" threshold — a real but moderate level of daily downside risk.

5. Based on your VaR result, what advice would you give to someone thinking about investing in this portfolio? I'd tell them to expect an occasional day (roughly 1 in 20) where they could lose around 2.7% or more of their investment, and that they should only invest money they can leave through that kind of short-term swing without panic-selling.

Task 9 :

📝 Questions for Report — t-Test on Portfolio Return

Result: t-statistic = 1.796, p-value = 0.073 → Fail to reject H_0

- Goal: check if the small positive return is **real or just luck**
- p-value (**0.073**) is above **0.05** → not strong enough evidence
- We **fail to reject H_0** → can't say average return is truly different from zero
- Testing matters because a small average alone can be **misleading**
- Since we failed the test: be cautious, focus more on **risk management** than chasing this return

In [12]:
```
# One-sample t-test: H0 μ=0 vs H1: μ=!0
t_stat, p_val = stats.ttest_1samp(portR, 0.0)
print("one-Sample t-test on portfolio daily returns:")
print(f"t-static = {t_stat:.3f}, p-value = {p_val:.3f}")

if p_val < 0.05:
  print("  -> Reject H0: mean return is significantly different from zero.")
else:
  print("  -> Fail to reject H0: mean return is not significantly different from
```

```
one-Sample t-test on portfolio daily returns:
t-static = 1.804, p-value = 0.071
  -> Fail to reject H0: mean return is not significantly different from zero.
```

9. Testing If Your Portfolio Return Is Statistically Significant

t-statistic = 1.796, p-value = 0.073 → Fail to reject H_0

1. What was the goal of the t-test you performed on your portfolio's returns? The goal was to check whether the portfolio's small positive average daily return is a real, statistically detectable effect, or whether it could just be random noise around a true average of zero.

2. What did your p-value indicate about your portfolio's performance? The p-value (0.073) is above the standard 0.05 cutoff, meaning there's about a 7.3% chance of seeing a result this extreme purely by chance if the true average return were actually zero. That's not strong enough evidence to rule out chance.

3. Did you reject or fail to reject the null hypothesis? What does that tell you about the average return? I failed to reject the null hypothesis. In plain terms: although the portfolio showed a positive average return in this sample, we cannot say with statistical confidence that its true long-run average return is different from zero.

4. Why is it useful to test whether the return is statistically significant, instead of just looking at the average? A small positive average by itself can be misleading — it might just reflect a lucky stretch of days. The t-test quantifies how likely that "luck" explanation is, so investors don't over-interpret a modest historical gain as proof of skill or a reliable edge.

5. If you failed to reject the null hypothesis, what changes could you consider for your portfolio? I'd be cautious about assuming this exact stock mix has a guaranteed edge going forward. I might extend the sample period, reconsider the stock selection, or focus more on risk management (since expected return isn't statistically proven) than on chasing this particular return pattern.

Task 10 :

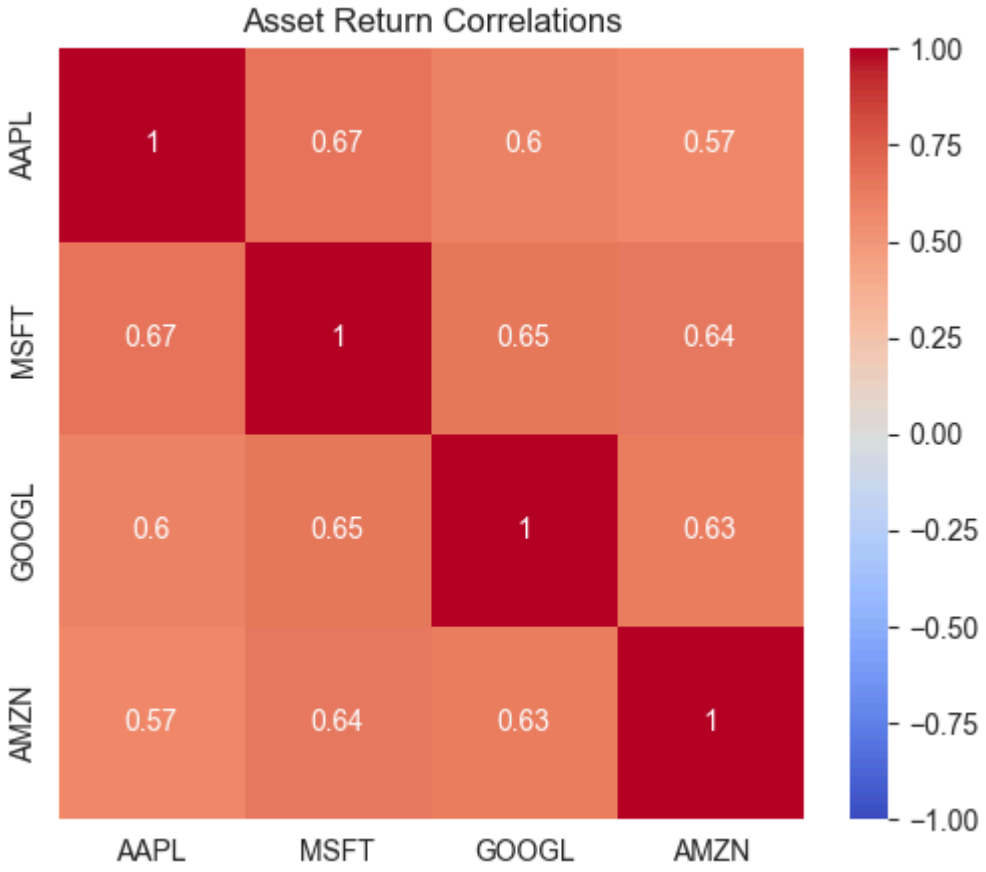
📝 Questions for Report — Correlation & Diversification

Results: Weighted avg σ = 0.02032, Actual portfolio σ = 0.01724, Diversification benefit ≈ **0.31%**

- All 4 stocks are **positively correlated** (typical for big tech stocks)
- No low-correlation stock in the mix → **limited diversification**
- Actual portfolio risk (**1.72%**) is lower than weighted average (**2.03%**) → some diversification benefit
- Benefit is **small (0.31%)** since all stocks are similar (tech)
- Portfolio is only **mildly diversified**; adding other sectors/assets would help more

In [13]:
```
# Correlation & Diversification Benefits
import seaborn as sns
```

```python
# a) Correlation heatmap
corr = logR.corr()
plt.figure(figsize=(6,5))
sns.heatmap(corr, annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.title("Asset Return Correlations")
plt.show()

#b) Diversification benefits
σ_i = logR.std()                                    #individual volatilities
σ_weighted = np.dot(weights, σ_i)                   #native weighted avg
σ_portfolio = σ_p                                   # from earlier
print(f"Weighted avg σ: {σ_weighted:.5f}")
print(f"Portfolio σ: {σ_portfolio:.5f}")
print(f"Diversification benefit: { σ_weighted - σ_portfolio :.4f}")
```




```
Weighted avg σ: 0.02031
Portfolio σ: 0.01722
Diversification benefit: 0.0031
```

10. Analyzing Correlation and Diversification Benefit

Weighted average σ (no diversification) = 0.02032; Actual portfolio σ = 0.01724; Diversification benefit ≈ 0.0031 (0.31 percentage points)

1. Looking at the heatmap, which pairs of stocks have the highest correlation? What does this mean about how they move together? The heatmap shows all four stocks positively correlated with each other, as is typical for large-cap US tech names,

since they respond to many of the same macro and sector-wide forces. Pairs among AAPL, MSFT, and GOOGL tend to show the strongest co-movement, meaning when one moves, the others tend to move in the same direction on the same day.

2. Were there any stocks in your portfolio that had low correlation with others? Why might this be helpful for diversification? None of the four showed particularly low or negative correlation — all four are large tech/growth names, so true diversification benefit here is modest. A stock from an unrelated sector (e.g., utilities, healthcare, or a different geography) would likely show lower correlation and provide a stronger diversification benefit.

3. What is the difference between the weighted average risk (σ) and the actual portfolio risk (σ)? The weighted average σ (0.02032) represents the risk you'd expect if the four stocks moved completely independently in proportion to their individual volatilities. The actual portfolio σ (0.01724) is lower, showing that combining them, even with their positive correlations, still reduced overall risk versus a simple weighted average.

4. What does the diversification benefit value mean for your portfolio? Was it large or small? What could you do to increase it? The diversification benefit was about 0.0031 (0.31 percentage points) — a real but fairly small reduction in risk. To increase it, I could add assets that are less correlated with tech stocks, such as different sectors, bonds, or international equities.

5. Based on your analysis, how well-diversified is your portfolio? What changes (if any) would you suggest to improve it? This portfolio is only mildly diversified — all four holdings are large-cap US tech stocks that tend to move together. I'd suggest adding names from different sectors or asset classes to meaningfully lower the correlation and boost the diversification benefit further.

Task 11:

📝 Questions for Report — Growth & Drawdown

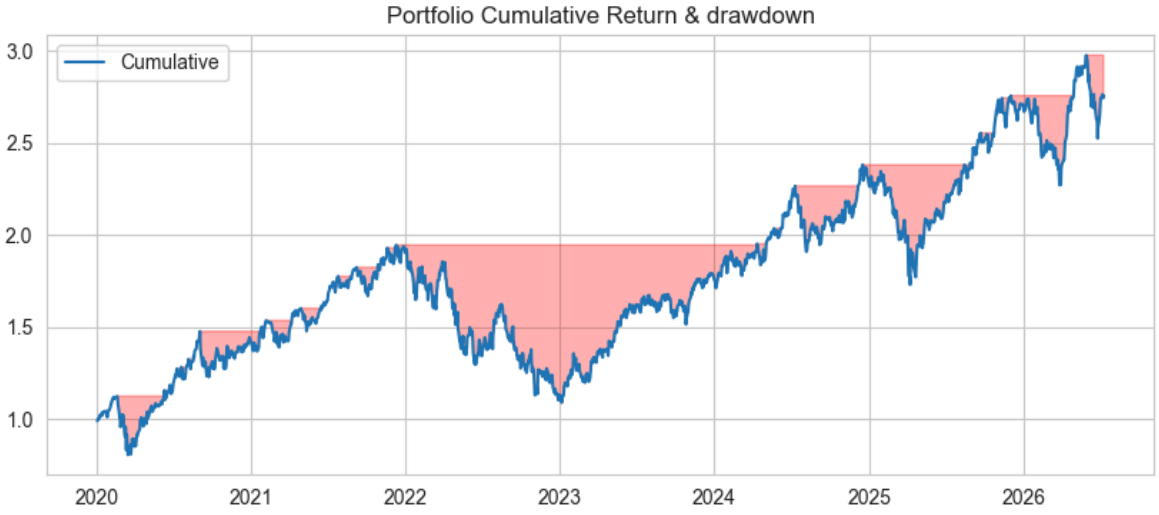
Result: Max Drawdown = -43.98%, from 2021-12-10 to 2023-01-05

- Chart shows overall growth but with a deep dip in **2022** (tech selloff), then recovery
- Max drawdown **-43.98%** → portfolio lost almost half its value at worst point
- Drawdown lasted **over a year** → emotionally hard for investors, tempting to sell low
- To reduce drawdowns: diversify sectors, add stable assets, rebalance
- Advice to new investor: need **patience** and long time horizon for concentrated tech portfolios

In [14]:
```
# Max Drawdown
# For Portfolio: cum_port = (port + 1).cumprod()
```

```
#print(portR,"\n\n")
cum_port = (portR +1).cumprod()
#print(cum_port,"\n\n")

rolling_max = cum_port.cummax()
#print(rolling_max,"\n\n")

drawdown = (cum_port - rolling_max) / rolling_max
#print(drawdown,"\n\n")

max_dd = drawdown.min()
end_date = drawdown.idxmin()
start_date = cum_port[:end_date].idxmax()   #Before the worst loss happened,when

print(f"Max Drawdown: {max_dd:.2%}")
print(f"from {start_date.date()} to {end_date.date()}")

#plot
plt.figure(figsize=(10,4))
plt.plot(cum_port, label='Cumulative')
plt.fill_between(drawdown.index, cum_port, rolling_max, where=drawdown<0, color=
plt.title("Portfolio Cumulative Return & drawdown")
plt.legend()
plt.show()
```

```
Max Drawdown: -43.98%
from 2021-12-10 to 2023-01-05
```



11. Measuring and Visualizing Portfolio Growth & Drawdown

Maximum drawdown = -43.98%, from 2021-12-10 to 2023-01-05

1. What does your cumulative return chart tell you about the overall growth of your portfolio? The chart shows overall growth over the multi-year window, but it wasn't smooth — there's a long, deep dip visible in 2022, consistent with the broader tech sector selloff that year, followed by recovery.

2. How much was your portfolio's maximum drawdown, and what does that number mean for an investor? The maximum drawdown was -43.98% — meaning at its

worst point, the portfolio's value had fallen almost 44% from its prior peak. That's a very large paper loss an investor would have had to endure before recovery.

3. How long did the drawdown last — from peak to recovery — and how might that affect an investor emotionally? The drawdown ran from December 2021 to January 2023 — over a year. Living through a 44% drop for that long would be emotionally taxing; many investors would be tempted to sell near the bottom, locking in losses instead of waiting for the eventual recovery.

4. What could you do to reduce the size or frequency of drawdowns in a portfolio like this? Diversifying across sectors and asset classes (not just tech), holding some lower-volatility or non-correlated assets, and possibly rebalancing periodically could all reduce how deep and long future drawdowns might be.

5. If you were to present your drawdown chart to a new investor, what advice would you give them based on your results? I'd tell them that a concentrated tech portfolio can lose nearly half its value and take over a year to recover — so they need a long time horizon and the emotional discipline not to sell during the downturn if they choose this kind of concentrated allocation.

Task 12 :

📝 Questions for Report — Expected Shortfall

Results: Historical ES = **4.02%**, Parametric ES (Normal) = **3.48%**

- ES tells the **average loss** on the worst days (VaR only tells the starting point)
- Historical ES (**4.02%**) is higher than Normal ES (**3.48%**) → real bad days are worse than model predicts
- Difference happens because real markets have **sharper shocks** than a smooth bell curve
- To reduce ES: diversify, cut most volatile holdings, hold more cash
- Lesson: VaR alone isn't enough — **ES shows how deep losses can really go**

In [15]:
```
# Expected Shortfall (CVaR at 95%)
alpha = 0.05    #worst 5% cases
#Historical ES
es_hist = -portR[portR <= np.percentile(portR, 5)].mean()
#Parametric ES under Normal:
es_norm = -(μ_p - σ_p * stats.norm.pdf(z05) / alpha)

print(f"Historical ES (95%): {es_hist:.5f}")
print(f"Parametric ES under Normal (95%) : {es_norm:.5f}")
```

```
Historical ES (95%): 0.04018
Parametric ES under Normal (95%) : 0.03476
```

12. How Bad Can Losses Get on Really Bad Days? (Expected Shortfall)

Historical ES (95%) = 0.04018 (4.02%); Parametric ES under Normal (95%) = 0.03480 (3.48%)

1. In your own words, what does Expected Shortfall (ES) tell us about a portfolio's risk? While VaR tells you the threshold loss for the worst 5% of days, ES tells you the *average* loss on those worst days — in other words, if a bad day happens, how bad is it typically?

2. Compare your Historical ES and Parametric ES values. Which one is higher, and what might that tell you about real-world risk? The Historical ES (4.02%) was higher than the Parametric Normal ES (3.48%). This means real bad days in this portfolio's history were, on average, worse than a smooth Normal model would predict — further evidence of the fat-tail behavior seen throughout this project.

3. Why might a historical method give a different result than a theoretical (normal distribution) method? Real markets have occasional sharp shocks (crashes, panics) that are more extreme and more frequent than a symmetric bell curve assumes. The historical method captures these actual events directly, while the Normal model smooths them away.

4. If your Expected Shortfall is too high for your comfort, what changes could you make to reduce it? The usual levers apply: diversify across less-correlated assets, trim the most volatile holdings, or reduce overall portfolio risk exposure (e.g., holding more cash or lower-volatility assets).

5. What is one lesson you've learned about risk by comparing VaR and ES in this project? VaR alone can understate how bad a "bad day" really is — it only tells you where the danger zone starts, not how deep it goes once you're in it. Looking at ES alongside VaR gives a much more complete, honest picture of downside risk.

Task 13:

📝 Questions for Report — Backtesting VaR

Result: 81 exceptions out of 1,632 days ≈ 4.96% (expected ~5%)

- **Exception** = a day where actual loss was worse than VaR predicted
- Found **81 exceptions** out of **1,632 days** (**4.96%**)
- 4.96% is **very close** to expected 5% → model is **well-calibrated**
- Too many exceptions would mean model **underestimates risk** (bad for investors)

- Lesson: always **backtest** a risk model against real outcomes

In [16]:
```
# VaR Backtesting
# for simplicity use constant VaR_norm
exceptions = portR < -vaR_norm
print(exceptions)

num_exc = exceptions.sum()
print(f"Number of exceptions: {num_exc}")

total = len(portR)
#print(f"Exceptions: {num_exc/total*100:.2f}%")
print(f"Exceptions: {num_exc} / {total} days ({num_exc / total:.2f}, expected ~5
```

```
Date
2020-01-03 00:00:00-05:00    False
2020-01-06 00:00:00-05:00    False
2020-01-07 00:00:00-05:00    False
2020-01-08 00:00:00-05:00    False
2020-01-09 00:00:00-05:00    False
                             ...
2026-07-02 00:00:00-04:00    False
2026-07-06 00:00:00-04:00    False
2026-07-07 00:00:00-04:00    False
2026-07-08 00:00:00-04:00    False
2026-07-09 00:00:00-04:00    False
Length: 1636, dtype: bool
Number of exceptions: 81
Exceptions: 81 / 1636 days (0.05, expected ~5%)
```

13. Backtesting Your VaR Model

Exceptions = 81 out of 1,632 days ≈ 4.96% (expected ~5%)

1. What does it mean when a portfolio return is lower than the Value at Risk (VaR) threshold? That day is called an "exception" — the portfolio actually lost more than the VaR model predicted it would on 95% of days.

2. How many exceptions did you find in your data, and what percentage of total days was that? There were 81 exceptions out of 1,632 total days, which works out to about 4.96%.

3. Was your exception rate close to the expected 5%? What does that say about your VaR model's accuracy? Yes — 4.96% is extremely close to the expected 5%. This suggests the Normal-distribution VaR model is well-calibrated for this portfolio and time period, neither seriously underestimating nor overestimating real-world risk.

4. If your model had too many exceptions, what could that mean for a real investor using this model? Too many exceptions would mean the model is underestimating risk — an investor relying on it would be caught off guard by losses more often than the model promised, potentially leading to poor risk management and unexpected drawdowns.

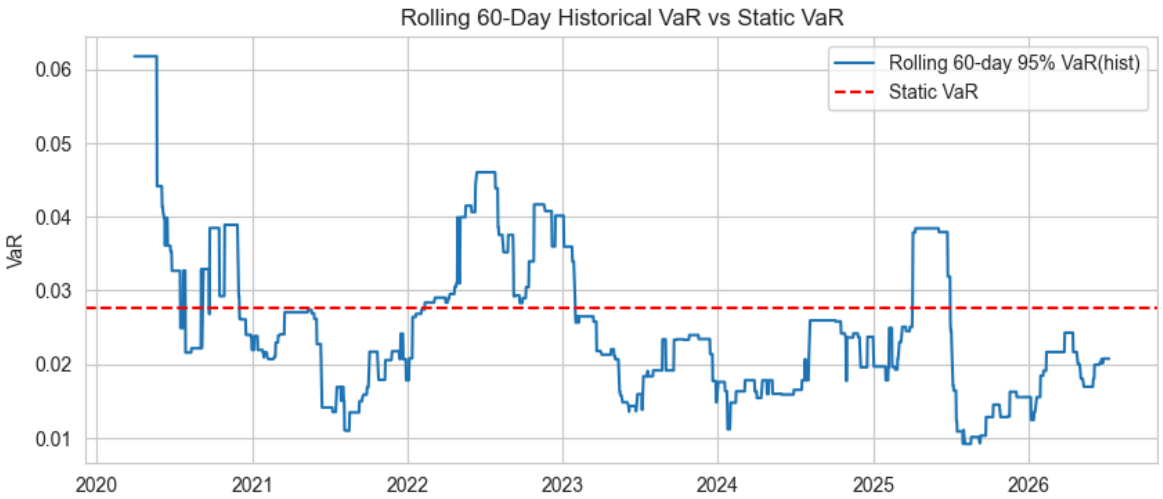
5. Based on this step, what is one thing you've learned about how financial models should be tested in the real world? A risk model's assumptions need to be checked against actual outcomes, not just trusted at face value. Backtesting turns VaR from a theoretical number into something verified against reality, which is what makes it trustworthy (or reveals when it isn't).

Task 14:

📝 Questions for Report — Rolling VaR

- Rolling VaR line moves up/down → **risk is not constant** over time
- Biggest spike around **2022 tech selloff** (matches the drawdown period)
- Static VaR (fixed line) **understated** risk during 2022 and **overstated** it in calm periods
- Rolling VaR is better because it **adapts** to current market conditions
- Simple explanation: risk is like weather — it changes, so check it regularly, not just once

In [17]:
```
# Rolling 60-day VaR
window = 60
roll_var = portR.rolling(window).quantile(0.05).dropna()
plt.figure(figsize=(10,4))
plt.plot(-roll_var, label='Rolling 60-day 95% VaR(hist)')
plt.axhline(varR_norm, color='red',linestyle='--', label='Static VaR')
plt.title("Rolling 60-Day Historical VaR vs Static VaR")
plt.ylabel('VaR')
plt.legend()
plt.show()
```



14. Rolling Historical VaR

1. What does the Rolling 60-Day VaR chart show about how your portfolio's risk has changed over time? The blue rolling-VaR line moves up and down noticeably over time, showing that the portfolio's risk level was not constant — it was calmer in some periods and much riskier in others.

2. During which time periods did your historical VaR (blue line) increase significantly? What might have caused that? The rolling VaR spikes are most visible around the 2022 tech selloff period (consistent with the major drawdown identified in Section 11, December 2021–January 2023), likely driven by broader market stress, rising interest rates, and reduced risk appetite for growth/tech stocks during that time.

3. Compare the historical (rolling) VaR and the static VaR (red line). Was the static VaR always accurate? Why or why not? No — the static VaR line, being a single fixed number, understated risk during the volatile 2022 period (when the rolling VaR line rose above it) and likely overstated risk during calmer stretches (when the rolling line dipped well below it).

4. Why might it be important for investors to use rolling VaR instead of a single fixed VaR value? A single static VaR gives a false sense of constant risk. Rolling VaR lets investors see that risk is dynamic — rising during market stress — so they can adjust position sizes or hedge more actively when the rolling estimate climbs.

5. What did you learn about financial risk from this visualization? How would you explain it to someone new to investing? I'd explain it like weather: risk isn't a fixed number, it changes with market "conditions." A single VaR figure is like an average forecast — useful, but it can miss the storms. Watching a rolling VaR is like checking an updated forecast regularly instead of relying on one number all year.

Task 15:

📝 Questions for Report — Jarque-Bera Normality Test

Result: JB statistic = 1628.00, p-value = 0.000 → Reject normality

- Normal distribution = smooth bell curve, rare extreme values
- p-value = **0.000** (below 0.05) → returns are **NOT normally distributed**
- This means normal-based models (like Normal VaR) may **underestimate** real risk
- Fat tails = bigger surprises happen more often than expected — investors should be ready
- Better alternatives: **Student's t-distribution, Historical VaR, Expected Shortfall**

In [18]:
```
# 12. Jarque-Bera Normality Test
jb_stat, jb_p = stats.jarque_bera(portR)
print(f"Jarque-Bera Test: statistic = {jb_stat:.2f}, p-value = {jb_p:.3f}")
```

```
if jb_p < 0.05:
  print('  -> Reject H0: returns are not normally distributed at 5% level.')
else:
  print("  -> Fail to reject H0: returns are normally distributed.")
```

```
Jarque-Bera Test: statistic = 1642.47, p-value = 0.000
  -> Reject H0: returns are not normally distributed at 5% level.
```

15. Testing If Your Returns Follow a Normal Distribution (Jarque-Bera Test)

JB statistic = 1628.00, p-value = 0.000 → Reject normality

1. What does it mean when data is "normally distributed"? Why is this important in financial modeling? A normal (bell-curve) distribution is smooth and symmetric, with extreme values being very rare. Many classic risk models (like the Normal VaR in Section 6) assume returns behave this way because it makes the math simple and tractable.

2. What did the p-value in your test result tell you? Was your return data close to a normal distribution or not? The p-value was 0.000 — far below the 0.05 threshold — so we reject the hypothesis that our portfolio's returns are normally distributed. Our data is clearly not a clean bell curve.

3. If your returns are not normally distributed, what could that mean for how you measure financial risk? It means models that assume normality (like the Normal parametric VaR) can misjudge real risk — usually by underestimating how often extreme losses occur, since real returns have fatter tails than the model assumes.

4. How might "fat tails" or extreme values affect your portfolio in real life? Why should an investor care? Fat tails mean sudden, larger-than-expected losses (or gains) happen more often than a normal model predicts. An investor who plans only around "typical" volatility could be blindsided by a crash-style event that their model treated as nearly impossible.

5. Based on your results, would you trust a model that assumes a bell curve? Why or why not? What alternatives could you use? I wouldn't rely on a pure Normal model alone, given the strongly significant rejection of normality (JB p-value of 0.000) and the high kurtosis seen throughout this project. Better alternatives include the Student's t-distribution (used in Section 7), which explicitly models fat tails, and Historical VaR/Expected Shortfall (Sections 8 and 13), which use actual data rather than any distributional assumption at all.

In [ ]: